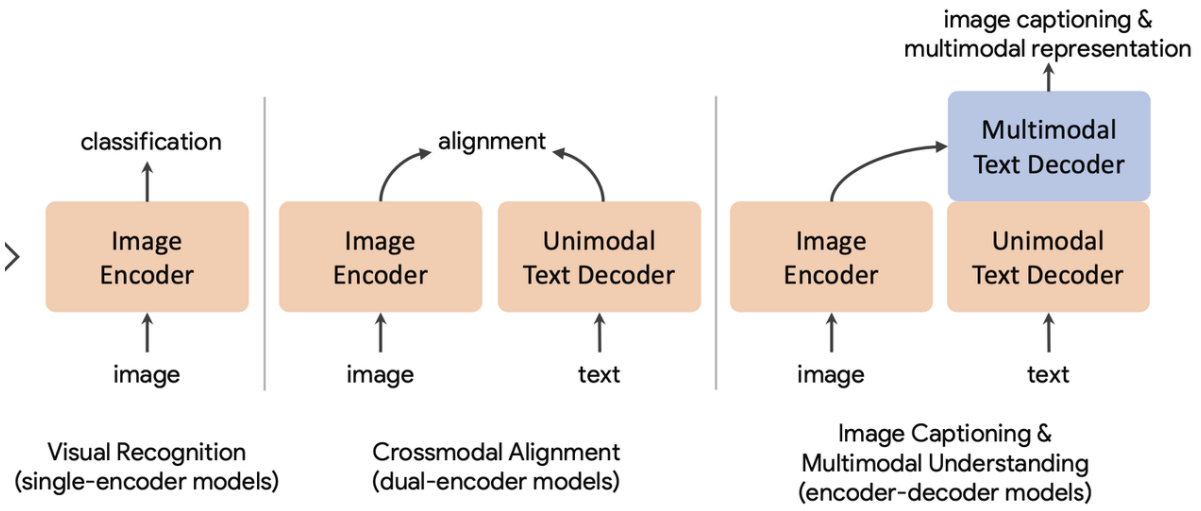
CoCa(2022, Google Research)

> CoCa：加快模型训练 (2022, Google Research)
> **Contrastive Captioners** are Image-Text Foundation Models



Zero-shot, frozen-feature or finetuning

用自然语言作为监督信号的三种基本方法：

Single(Fusion)-encoder model	Dual-encoder model	Encoder-decoder model
在**手工标注的干净的数据集**上通过图像分类任务预训练一个**视觉编码器**，训练损失为交叉熵损失。只能去做一些视觉方面的任务，并不能做多模态任务。	不再需要手工标注的干净的数据集，而是**借助大量带噪声的网络上的图像文本对**，通过对比损失预训练 **Dual-Encoder 的视觉模型和语言模型。** 在一些下游任务中，比如 zero-shot 图像分类或者图文检索任务，可以获得很好的效果。但是这种方法图像和文本独立编码，且编码过程中并没有任何图像和文本侧的交叉，只在最后计算 cosine 相似度，缺少图像和文本的融合表示，因此不适用于 VQA 等需要对图像和文本联合理解的任务	**生成式任务**，也叫描述任务，要求**模型以自回归的方式根据上文来推测下文是什么。**模型的架构是 **Encoder-Decoder 类型**，这种类型的模型架构可以**联合学习视觉和多模态的特征。**在预训练期间，模型在 Encoder 端 (可以使用 Vision Transformer 或 ConvNet) 获取图像并编码图片的特征。Text Decoder 输出上使用语言建模 (LM) 损失函数，推测下文的输出。同时，解码器的输出也可以作为视觉语言的联合表征。

1. 概述

CoCa 是一种多模态 Transformer 模型，顾名思义**Contrastive Captioners**，就是在训练的时候使用两个目标函数，分别是**基于对比学习的损失函数 Contrastive Loss（ITC）和基于生成式图像字幕任务的目标函数 Captioning Loss（LM）。**CoCa 融合了上面 3 种模型结构，既能生成图像侧和文本侧独立的表示（CLIP），又能进行更深层次的图像、文本信息融合以及文本生成（Encoder-decoder），适用于更广泛的任务：包括 Image Encoder，单模态 Text Encoder 和多模态 MultiModal Encoder。

也可以简单认为 CoCa 是 CLIP 加上了一个 image caption 的预训练任务。Coca 因此具备了三种能力，纯图像

模型的表示生成，图文多模态表示对齐，图片描述生成 (image caption)。

其实用单模态文本特征输入对齐图像特征的方式这种方法在 ALBEF 中就使用过，类似也是先用对比损失对齐文本和图像特征，然后使用 cross attention 来辅助解码语言模型的预训练，不过 CoCa 并没有 ALBEF 的引用。其实 CoCa 可以算是 ALBEF 的一个后续工作。

主要贡献点：

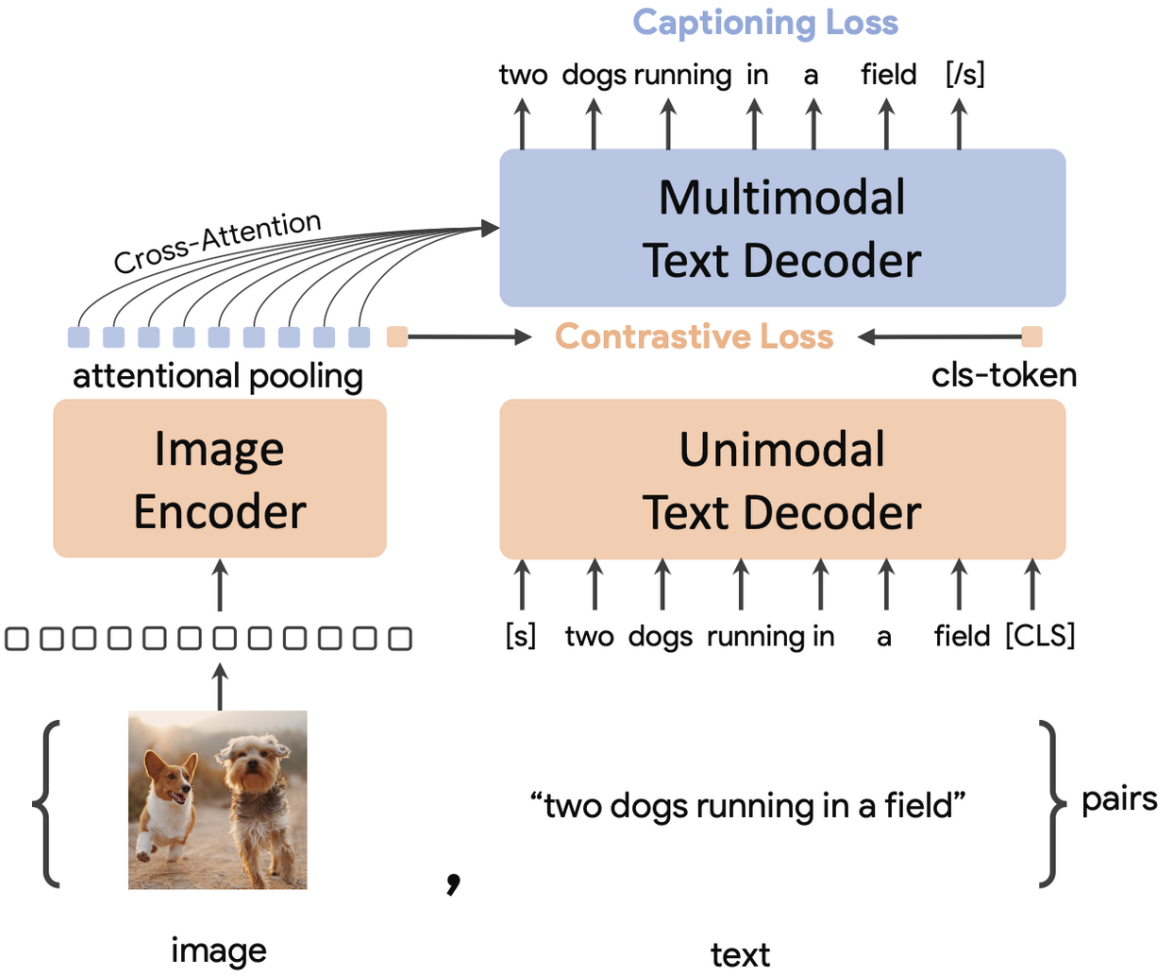
1. 融合了三种基本模型结构的特点，**统一了对比学习和生成任务**，通过共享的编码器和解码器框架完成多任务学习，生成更为准确且多样的图像描述
2. 加快了训练速度，处理一个 batch 的图像-文本对**只需要前向和后向计算一次**（ALBEF 需要两次）。

2. 模型架构

CoCa 以上面三种基础结构的优势为基础，使用带有解耦解码器的单个 encoder-decoder Transformer 来处理对比学习信号和生成学习信号。

CoCa 的整体结构包括 3 个部分：**一个图像 encoder（Image Encoder）和两个文本 decoder（Unimodal Text Decoder、Multimodal Text Decoder）。Image Encoder 端还有一个 Attentional Pooling。**

> 整个结构看作是一个 encoder-decoder Transformer，encoder 端是视觉的，decoder 端是文本的，然后有将 decoder 切分成 2 部分，后面的部分有 cross attention，前面部分没有。



1. Image Encoder 图像编码器
   类似于 Single-encoder 模型的工作方式

- 使用一个视觉变换器（ViT，默认为 ViT，也可以使用其他 CNN）对输入图像进行编码，生成一系列图像特征表示 img_embed
- img_embed 通过**注意力池化 attentional pooling**进一步提取，分别生成两个特征：
  - **对比特征 (Contrastive Feature, con_feature)**：专门的 **[cls]token** ，用于对比学习，通过与文本的 [cls] 特征进行对比。
  - **描述特征 (Captioning Feature, cap_feature)**：除了 [cls]，其他的图像特征经过 attentional pooling 后，传入多模态 Text Decoder 中，用于文本生成任务，作为多模态解码器的输入。

> **attentional pooling**
> (源自于 Set Transformer，其实作用类似于 BLIP-2 中的 Q-Former，作为 bottleneck，将信息进行进一步的提取，把最有用的信息融合在更少的 token 中)
> **设计原因**：image-encoder 侧的信息既可以提取一个统一的 [cls] 表示，也可以提取每个 token 的表示，二者各有不同的适用场景（单个 [cls]token 可作为视觉识别任务的全局表示，而使用更多视觉 tokens 可提供更精细的细节，这对于需要区域级特征的多模态任务很有帮助），所以 CoCa 采用特定于任务的 attentional pooling 来为不同的训练目标和下游任务定制**视觉表示**。
> **原理**：可学习的，由 **multi-head attention** 实现，用于对 image-encoder 的输出进行处理，根据特定任务进行特征聚合。其中，query 是预先定义的一个可训练的 embedding（根据不同任务，这个 query 不同），key 和 value 是 image encoder 的输出（img_embed）。
> - 对于**对比学习**任务，采用**维度为 1 的 query** 进行 attention，将 img_embed 融合为**单个向量表示 [cls]**，表示**图像全局信息**，以支持与文本端文本全局特征的对比学习任务。
>
>   con_feature = attentional_pooling(img_feature, con_query)
>
> - 对于**生成任务**，采用**维度为 256 的 query** 进行 attention，将 img_embed 生成**更多的视觉 tokens 表示更多精细的细节**，以此和文本特征进行多模态融合。
>
>   cap_feature = attentional_pooling(img_feature, cap_query)

2. Unimodal Text Decoder 单模态文本解码器
   - 处理纯文本信息（不参与对图像特征的 cross-attention，不受图像影响）
   - 文本输入时添加一个 [cls]token，输入到 tranformer 中进行编码，输出一个 text_embed，其中最后一个 [cls] 代表文本的全局信息。
   - 这个文本 [cls] 与上面的图片 [cls] 用于 ITC loss，然后 text_embed 其余部分输入到 Multimodal Text Decoder 用于融合多模态信息。
   - ⚠ 单向 decoder，用的是 casual attention（Masked），为防止信息泄漏，只能看到序列前面的部分
3. Multimodal Text Decoder 多模态文本解码器
   - 输入：**图像的描述特征（cap_feature）+ 上一步 text_embed（除去 [cls]）**
   - 利用 **cross attention**，**生成与图像内容相关的描述（Caption）（/还原对应文本）**，实现对视觉和语言的联合理解。
   - 输出文本序列，优化目标为**描述生成损失 (Captioning Loss)——LM loss。**
   - ⚠ 单向 decoder，用的是 casual attention（Masked），为防止信息泄漏，只能看到序列前面的部分

⚠

1. 文本端，Unimodal Text Decoder 和 Multimodal Text Decoder 的**层数是相同的**，也就是把一个 transformer 解码器平均分为两部分，因为浅层的早期融合对于多模态理解也可能是不必要的。
2. CoCa 中 Decoder 部分所有的**自注意力都是用了掩码操作**的，对于一个批量的图像文本对，只进行**一次**前向传播和反向传播，和一些需要多阶段训练的模型相比，这就带来了效率上的提升。

CoCa 和 ALBEF 其实是很像的，他们都采用“图像分支 + 文本分支”的结构，左边是**图像编码器** (Image Encoder)，右边是**文本处理器** (Text Branch)。都是在文本分支中的后一部分使用 cross attention 将图像信息和

文本信息进行融合。

但也是文本分支中，ALBEF 使用 Encoder 来处理，CoCa 则使用 Decoder 来处理。

> Decoder 的意义：CoCa 的 Decoder 更倾向于语言生成任务（LM），不仅能抽取文本特征，还能自然支持语言生成任务（如 Captioning）。
> Encoder 的意义：ALBEF 的 Encoder 主要用于文本特征的理解，适用于 ITC 任务。

所以 ALBEF 在每个 iteration 训练的时候，文本部分的模型要 forward 2 次，一次是输入完整的文本做图文匹配任务，一次是输入 mask 之后的文本做完形填空任务。

但是 CoCa 只需要 1 次前向和反向，即可同时计算 ITM loss 和 LM loss。

解耦解码器的作用：

> CoCa 中**解耦的解码器结构（decoupled decoder architecture）**：
> 解耦：在模型内部对不同任务的功能与特征进行明确分离的设计方式
>
> 1. **输入信息的解耦**：
>   单模态解码器与多模态解码器分别处理不同类型的信息，前者纯粹用于文本特征提取（对比任务），后者用于视觉-文本的联合理解与生成任务。
> 2. **训练目标的解耦**：
>   单模态解码器产生的文本特征用于对比学习任务，而多模态解码器则专注于生成任务（如图像描述）。两个解码器可以独立地计算不同的损失函数，这样两类任务的训练就能高效且互不干扰地进行。

高效性：模型只需一次前向传播就能同时完成对比任务和生成任务，而无需分别进行多次独立计算。
灵活性：两个子解码器能分别针对各自任务优化，不会因信息互相干扰而降低性能。
泛化性：单模态文本解码器学到纯粹的文本表示，而多模态解码器则能够有效地融合图像和文本的表示，从而提升 CoCa 在不同任务场景下的泛化能力。

3. 预训练

在训练过程中，CoCa 采用了两个个核心的训练目标：ITC、LM

CoCa 综合训练目标：$L_{CoCa} = \lambda_{Con} L_{Con} + \lambda_{Cap} L_{Cap}$，其中：$\lambda_{Con}$ 与 λ_{Cap} 是用于调节 ITC loss 和 LM loss 之间权重的超参数。

- **双编码器对比学习目标（Dual-Encoder Contrastive Learning）**：**ITC loss**
  用于优化**图像编码器**和**单模态文本解码器**在多模态嵌入空间中的对齐。
  - **从头开始训练**
  - who 参与 ITC 对比学习：
    - image-encoder 的输出经过一个 Attentional Poolling 后输出的**单个图像 [cls] token**
    - Unimodal Text Decoder 的输出中最后的单个**文本 [cls] token**
- **编码器-解码器描述生成目标（Encoder-Decoder Captioning）**：**LM loss**（BLIP、GPT 系列用的 Language Modeling Loss）
  用于优化**图像编码器**和**多模态文本解码器**对生成任务（文本生成）的质量，使模型学习如何结合图像与文本特征，准确地进行图像描述生成和多模态任务的理解。
  LM loss：在已知图像特征和之前生成的所有词语的情况下，计算生成每个词语的负对数似然值（negative log-likelihood）。通过最小化该损失，CoCa 能够逐步学会生成准确且连贯的图像描述。
  - **从头开始训练**
  - who 参与 LM 生成任务：
    - image-encoder 的输出经过一个 Attentional Poolling 后输出的**图像特征序列**

- Unimodal Text Decoder 输出的除去最后的 [cls]token 的**文本特征序列**

总结 CoCa 训练过程：

1. 图像通过 Image Encoder（ViT），得到一系列的图像特征 token；

2. 对图像特征进行 attention pooling（multihead attention），取第 0 位作为计算对比损失的 [cls]token，后 255 位作为计算生成损失的视觉 token；

3. 文本（在最后添加 [cls]token）通过文本的解码器，得到一系列的文本特征 token。

4. 图像的 [cls]token 和文本的 [cls]token 计算 ITC loss

5. 图像的视觉 token 和文本的其他 token 传到多模态的 Text Decoder 里做 Cross Attention，融合视觉和文本的特征。多模态的特征用 Captioning Loss（LM）训练。

伪代码：

代码块

```
# 输入数据说明：
# image：输入图像数据
# text.ids：输入文本对应的token id序列
# text.labels：文本的目标标签（用于caption生成的真实标签）
# text.mask：文本掩码，表示有效token位置
# con_query：对比学习的查询向量（用于生成对比特征）
# cap_query：图像描述任务的查询向量（用于生成caption特征）
# cls_token_id：词表中特殊的[CLS] token的id

# 注意力池化函数定义：
def attentional_pooling(features, query):
    # 使用多头注意力机制，以query为查询向量对特征进行池化
    out = multihead_attention(features, query)
    # 对注意力输出进行Layer Normalization，稳定训练过程
    return layer_norm(out)

# 将图像通过视觉Transformer（ViT）编码器，得到图像特征
img_feature = vit_encoder(image)  # 输出形状：[batch_size，序列长度，特征维度]

# 利用对比学习的单个查询向量（con_query）对图像特征进行池化，获得用于对比任务的图像表示
con_feature = attentional_pooling(img_feature, con_query)  # [batch_size，1，特征维度]

# 利用图像描述任务的多个查询向量（cap_query）对图像特征进行池化，获得用于caption任务的图像表示
cap_feature = attentional_pooling(img_feature, cap_query)  # [batch_size，N，特征维度]

# 将文本的token序列和一个特殊的[CLS] token id拼接
ids = concat(text.ids, cls_token_id)

# 构造对应文本掩码，其中CLS token位置不需要额外mask（这里用0填充）
mask = concat(text.mask, zeros_like(cls_token_id))  # 去除CLS token的padding影响

# 将文本id序列转化为文本嵌入向量
txt_embs = embedding_lookup(ids)

# 单模态文本解码器（Unimodal Text Decoder）：仅对文本嵌入进行Transformer编码，不考虑图像特征
unimodal_out = lm_transformers(txt_embs, mask, cross_attn=None)
```

```
# 多模态文本解码器（Multimodal Text Decoder）：融合文本嵌入和图像特征（cap_feature）
# 输入时排除最后一个CLS token对应的输出
multimodal_out = lm_transformers(
    unimodal_out[:, :-1, :],  # 不包含最后一个CLS token的文本嵌入
    mask,
    cross_attn=cap_feature    # 与图像特征进行跨注意力融合
)

# 提取单模态文本解码器输出的最后一个token（即CLS token）特征，并进行Layer Normalization
cls_token_feature = layer_norm(unimodal_out)[:, -1:, :]  # [batch_size, 1, 特征维度]

# 计算对比学习损失（contrastive_loss），将图像特征(con_feature)和文本的CLS token特征进行对比
con_loss = contrastive_loss(con_feature, cls_token_feature)

# 计算图像描述（caption）任务的交叉熵损失（softmax cross entropy loss）
cap_loss = softmax_cross_entropy_loss(
    multimodal_out,
    labels=text.labels,  # caption任务对应的目标标签
    mask=text.mask       # mask用于指示有效的token位置
)

# 总体损失由以上两个损失组合而成
```

4. 模型实现代码

Image Encoder

代码块

```
    def _encode_image(self, images, normalize: bool = True):
        image_latent, tokens_embs = self.visual(images)
        image_latent = F.normalize(image_latent, dim=-1) if normalize else image_latent
        # image_latent:constractive_token, tokens_embs: caption_token
        return image_latent, tokens_embs

    #### self.visual(images):
    def forward(self, x: torch.Tensor):
        # [b, 3, 224, 224]--->[b, 1024, 16, 16]
        x = self.conv1(x)
        # [b, 1024, 16, 16]--->[b, 1024, 256]
        x = x.reshape(x.shape[0], x.shape[1], -1)
        # [b, 1024, 256]--->[b, 256, 1024]
        x = x.permute(0, 2, 1)

        # 在序列长度上给图像嵌入一个类别，x: [b, 256 + 1, 1024]
        x = torch.cat([_expand_token(self.class_embedding, x.shape[0]).to(x.dtype), x], dim=1)
        # 嵌入位置编码，x: [b, 256 + 1, 1024]
        x = x + self.positional_embedding.to(x.dtype)
        # patch_dropout，  x: [b, 256 + 1, 1024]
        x = self.patch_dropout(x)
        # LayerNorm处理 x: [b, 256 + 1, 1024]
        x = self.ln_pre(x)
        # NLD -> LND [b, 256 + 1, 1024]---> [256 + 1, b, 1024]
        x = x.permute(1, 0, 2)
        # transformer网络处理
        x = self.transformer(x)
```

```
        # LND -> NLD  [256 + 1, b, 1024]--->[b, 256 + 1, 1024]
        x = x.permute(1, 0, 2)

        if self.attn_pool is not None:
                        # this is the original OpenCLIP CoCa setup, does not match paper
                        # x: [b, 257, 1024]--->[b, 256, 768]
                        x = self.attn_pool(x)
                        # ln归一化, [b, 256, 768]
                        x = self.ln_post(x)
                        # pooled: 类别token: [b, 768] tokens: 图像token:[b, 255, 768]
                        pooled, tokens = self._global_pool(x)

        # pooled: [b, 768]@[768, 768]--->[b, 768]
        if self.proj is not None:
            pooled = pooled @ self.proj
        # 同时返回cls-token和视觉token
        if self.output_tokens:
                         return pooled, tokens

        return pooled
```

Attentional Pooling

代码块

```
# self.attn_pool(x)
class AttentionalPooler(nn.Module):
    def __init__(
            self,
            d_model: int,
            context_dim: int,
            n_head: int = 8,
            n_queries: int = 256,
            norm_layer: Callable = LayerNorm
    ):
        super().__init__()
        self.query = nn.Parameter(torch.randn(n_queries, d_model))
        self.attn = nn.MultiheadAttention(d_model, n_head, kdim=context_dim, vdim=context_dim)
        self.ln_q = norm_layer(d_model)
        self.ln_k = norm_layer(context_dim)

    def forward(self, x: torch.Tensor):
        # ln归一化, NLD -> LND [b, 257, 1024]--->[257, b, 1024]
        x = self.ln_k(x).permute(1, 0, 2)
        N = x.shape[1]
        # q: [256, 768]
        q = self.ln_q(self.query)
        # q: [256, 768]--->[256, 1, 768]--->[256,b, 768], k=v=x, x:[257, b, 1024]
        # out: [256, b, 768],  MultiheadAttention
        out = self.attn(q.unsqueeze(1).expand(-1, N, -1), x, x, need_weights=False)[0]
        # out: [256, b, 768]--->[b, 256, 768]
        return out.permute(1, 0, 2)  # LND -> NLD
```

Unimodal Text Decoder

代码块

```
    def _encode_text(self, text, normalize: bool = True):
        # text_latent:[b, 768], token_emb:[b, 76, 768]
```

```
        text_latent, token_emb = self.text(text)
        text_latent = F.normalize(text_latent, dim=-1) if normalize else text_latent
        return text_latent, token_emb

    def forward(self, text):
        cast_dtype = self.transformer.get_cast_dtype()
        seq_len = text.shape[1]
        # x: [b, 76, 768]， 将text: [b, 76]进行embeding， F.embedding(text, weight=[40408, 768])
    49408---一共49408个单词，每个单词维度768
        x = self.token_embedding(text).to(cast_dtype)
        attn_mask = self.attn_mask
        if self.cls_emb is not None:
            seq_len += 1
            # 在文本token末尾嵌入cls_token, x:[b, 76, 768] ---> [b, 76+1, 768]
            x = torch.cat([x, _expand_token(self.cls_emb, x.shape[0])], dim=1)
            # cls_mask: [12b, 77, 77]， text: [b, 76]
            cls_mask = self.build_cls_mask(text, cast_dtype)
            # 将单词有序遮挡mask， attn_mask: [[0, -inf， -inf，...-inf], [0, 0, -inf, ..., -inf
    ],...[0, 0, 0,...,0,-inf], [0, 0, 0, ...,0]]
            if attn_mask is not None:
                # attn_mask: [1,77, 77] + cls_mask[12b, 77, 77] ===> 获得最终的attn_mask: [12b,
    77, 77]，有单词的位置为0，被遮挡以及没单词的位置为-inf
                attn_mask = attn_mask[None, :seq_len, :seq_len] + cls_mask[:, :seq_len, :seq_len]
        # 加上位置编码， x: [b, 77, 768]
        x = x + self.positional_embedding[:seq_len].to(cast_dtype)
        # x: [b, 77, 768]--->[77, b, 768]
        x = x.permute(1, 0, 2)  # NLD -> LND
        # 进入transformer学习， x: [77, b, 768]
        x = self.transformer(x, attn_mask=attn_mask)
        # x: [77, b, 768]--->[b, 77, 768]
        x = x.permute(1, 0, 2)  # LND -> NLD

        # x.shape = [batch_size, n_ctx, transformer.width]
        if self.cls_emb is not None:
            # presence of appended cls embed (CoCa) overrides pool_type, always take last token
            # pooled: cls_token: [b, 768] tokens: 图像token:[b, 76, 768]
            pooled, tokens = text_global_pool(x, pool_type='last')
            # layernorm
            pooled = self.ln_final(pooled)  # final LN applied after pooling in this case
        # [b,  768] @ 【768, 768】---> [b, 768]
        pooled = pooled @ self.text_projection
        # pooled: [b, 768], tokens:[b, 76, 768]
        if self.output_tokens:
            return pooled, tokens
        return pooled
```

代码块

```
    def build_cls_mask(self, text, cast_dtype: torch.dtype):
        # 找到text中存在单词的cls_mask，值设为True，text:[b, 76], cls_mask: [b, 1, 76]
        cls_mask = (text != self.pad_id).unsqueeze(1)
        # cls_mask: [b, 1, 76]--->[b, 77, 77]
        cls_mask = F.pad(cls_mask, (1, 0, cls_mask.shape[2], 0), value=True)
        # 随机一个[b, 77, 77]的mask
        additive_mask = torch.empty(cls_mask.shape, dtype=cast_dtype, device=cls_mask.device)
        # 全部填充为0 additive_mask:[b, 77, 77]
        additive_mask.fill_(0)
```

```
        # 不满77长度的单词中，0填充的位置换为-inf
        additive_mask.masked_fill_(~cls_mask, float("-inf"))
        # 将additive_mask在batch维度上重复self.heads (12) 次，[b, 77, 77]--->[12b, 77, 77]
        additive_mask = torch.repeat_interleave(additive_mask, self.heads, 0)
        return additive_mask
```

Multimodal Text Decoder

代码块

```
# logits: [b, 76, 49408], image_embs:caption_embedings[b, 255, 768], token_embs：文本embedings [b
    , 76, 768]
logits = self.text_decoder(image_embs, token_embs)
# self.text_decoder forward
def forward(self, image_embs, text_embs):
    # [b, 76, 768]--->[76, b, 768]
    text_embs = text_embs.permute(1, 0, 2)
    # [b, 255, 768]--->[255, b, 768]
    image_embs = image_embs.permute(1, 0, 2)
    # 76
    seq_len = text_embs.shape[0]
    # cross-attention: q=text_embs, k_x=image_embs, v_x=image_embs
    for resblock, cross_attn in zip(self.resblocks, self.cross_attn):
        text_embs = resblock(text_embs, attn_mask=self.attn_mask[:seq_len, :seq_len])
        # q=text_embs, k_x=image_embs, v_x=image_embs
        text_embs = cross_attn(text_embs, k_x=image_embs, v_x=image_embs)
    # x: [76, b, 768]--->[b, 76, 768]
    x = text_embs.permute(1, 0, 2)  # LND -> NLD
    # layer_norm
    x = self.ln_final(x)
    # x: [b, 76, 768] @ [768, 49408] ---> [b, 76, 49408]
    if self.text_projection is not None:
        x = x @ self.text_projection
          # [b, 76, 49408]
    return x
```

Loss 计算

代码块

```
    def forward(self, image_features, text_features, logits, labels, logit_scale, output_dict=
    False):

        clip_loss = torch.tensor(0)
        # constractive loss
        if self.clip_loss_weight:
            # image_features: [b, 768], text_features:[b, 768], logit_scale:温度系数
            clip_loss = super().forward(image_features, text_features, logit_scale)
            clip_loss = self.clip_loss_weight * clip_loss
                        # caption loss, self.caption_loss:CE loss
        caption_loss = self.caption_loss(
            logits.permute(0, 2, 1), # [b, 76, 49408]
            labels, # [b, 76]
        )
        caption_loss = caption_loss * self.caption_loss_weight

        if output_dict:
            return {"contrastive_loss": clip_loss, "caption_loss": caption_loss}
```

```
        return clip_loss, caption_loss

                # clip_loss
    def forward(self, image_features, text_features, logit_scale, output_dict=False):
        device = image_features.device
        # 假设有N个图像-文本对: logits_per_image: [N, N], logits_per_text: [N, N]
        logits_per_image, logits_per_text = self.get_logits(image_features, text_features,
     logit_scale)
        # 假设有N个图像-文本对: labels=[0, 1, 2,....N]
        labels = self.get_ground_truth(device, logits_per_image.shape[0])
        # 总损失 = （图像维度的损失 + 文本维度的损失）/ 2
        total_loss = (
            F.cross_entropy(logits_per_image, labels) +    # 图像维度的损失
            F.cross_entropy(logits_per_text, labels)       # 文本维度的损失
        ) / 2

        return {"contrastive_loss": total_loss} if output_dict else total_loss
```

5. 总结

CoCa 既简单又能高效训练，同时具有多种模型能力：

- **高效损失计算**
  CoCa 处理一个 batch 的图像-文本对只需要**前向和后向计算一次**，在一次前向传播中同时生成用于对比损失和生成损失的输出，**计算量很少**（从而导致可以从头训练，因为不耗时），这使得 CoCa 能够在两种损失之间共享大部分计算量，相比标准的编码器-解码器模型，仅增加极少的额外计算开销。
- **端到端预训练**
  CoCa 在两个目标函数上**从头开始训练**。使用**多种数据源**，包括标注图像和带有噪声的替代文本（alt-text）图像。无论是对比学习目标还是生成学习目标，所有标签都统一作为文本处理。
- **零样本迁移**
  优化生成损失的编码器结构可**直接用于图像描述生成和 zero-shot 学习**。
- **冻结特征评估**
  CoCa 采用**任务特定的注意力池化（attentional pooling）**，为不同任务定制视觉表示，同时保持主干编码器不变。这样，它能够作为冻结编码器高效运行，仅需训练一个新的池化器（pooler）用于特征聚合。这种方法对于多任务学习尤其有利，因为多个任务可以共享相同的冻结图像编码器，同时各自使用不同的任务特定头部。

由于 CoCa 具备这种统一的架构，使得它能够在不同任务之间实现灵活的迁移，只需稍作调整即可完成各类下游任务。具体而言，**CoCa 模型特别适用于以下任务场景**：

- **视觉识别任务（Visual Recognition）**
  仅使用图像编码器部分，CoCa 便能完成图像分类和目标检测等任务。
- **跨模态对齐任务（Crossmodal Alignment）**
  通过图像编码器和单模态文本解码器，CoCa 能够有效完成图文检索以及零样本（Zero-shot）分类等任务。
- **图像描述及多模态理解任务（Image Captioning & Multimodal Understanding）**
  使用完整的编码器-解码器结构，CoCa 能够准确地为图像生成描述，并回答与视觉内容相关的问题。

Coca 的模型实现并不难，但是想复现它难度非常大。原因是：

- **模型大**：虽然很简单，但它训练出来最大的模型参数量已经达到了 2.1 billion，算是视觉或者多模态里面非常大的一个模型（当然在 NLP 那边已经有几十亿上百亿的模型）

- **训练的数据集**:作者不只用了之前训练 Align 用的多模态的数据集，同时还把 GFT 3 billion（google 私有数据）图像分类的数据集转化成了多模态数据集，加在一起有几十亿的训练数据，所以不论是模型还是这个数据都远超之前所有工作的这个 scale，效果也是非常明显的。